ESCUELA DE ADMINISTRACIÓN Y NEGOCIOS

Cobertura de testing

Evaluación 2 (30%) — DSY1104

Tienda online El Rincón de la Tía Ceci

Integrantes:

Carolina Nataly Hermosilla Baeza

Pedro Felipe Toledo Jaramillo

Asignatura: DSY1104 — Evaluación 2 (30%)

Sección: completar

Docente: completar

Fecha: 26-10-2026

Versión 1.0

Proyecto: Transformación de front-pos a Next.js

Asignatura: DSY1104 — Evaluación 2 (30%)

Integrantes: Carolina Nataly Hermosilla Baeza · Pedro Felipe Toledo Jaramillo

Sección: completar

Docente: completar

Fecha: 26-10-2026

Versión: 1.0

---

1. Objetivo

Documentar la estrategia, las herramientas, los casos de prueba y los resultados obtenidos al probar la tienda online y su panel administrativo. El objetivo es verificar la lógica de los datos, las operaciones CRUD, el carrito, las validaciones y el comportamiento de componentes y formularios.

2. Herramientas utilizadas

Herramienta	Versión	Uso
Jasmine	5.x	Framework de pruebas (describe, it, expect, spies).
Karma	6.4.4	Runner de las pruebas.
karma-jasmine	5.1.0	Integración de Jasmine con Karma.
karma-webpack	5.0.1	Empaquetado de los archivos de prueba.
Babel	7.x	Transformación de JSX y módulos ES.
React Testing Library	16.x	Renderizado de componentes y simulación de eventos.
jsdom	30.x	Entorno de navegador para la ejecución alternativa.
ChromeHeadless	Navegador instalado	Ejecución principal de Karma.

3. Configuración

• karma.conf.js: frameworks, archivos, preprocesadores, alias de Webpack, launcher de ChromeHeadless y launcher de jsdom.

• tests/babel.config.js: presets de entorno y de React con el runtime automático, usado solo por las pruebas.

• tests/setup.js: limpieza de Testing Library, localStorage en memoria y fetch de respaldo.

• tests/mocks/: reemplazos de next/link y next/navigation para poder probar los componentes sin un servidor de Next.

4. Estrategia

Se combinaron tres niveles:

1. Pruebas unitarias de lógica: funciones puras de validación, cálculo de totales y utilidades.

2. Pruebas de acceso a datos: se simula fetch para verificar método, URL y

cuerpo de cada operación CRUD.

3. Pruebas de componentes: se renderizan componentes y formularios con React Testing Library, se disparan eventos y se verifican props, estado y llamadas.

No se realizaron pruebas E2E con navegador real dentro de este entorno porque el sandbox de ejecución bloquea la creación de procesos e IPC de Chromium. La configuración deja npm test con ChromeHeadless para el equipo y npm run test:jsdom como alternativa verificada.

5. Casos de prueba

5.1 tests/api.test.js — módulo de API

ID	Caso	Entrada	Resultado esperado	Estado
API-01	Listar productos	listarProductos()	Devuelve el arreglo y llama GET /productos con cache: no-store.	Exitosa
API-02	Crear producto	crearProducto({...})	Llama POST / productos y envía el cuerpo JSON.	Exitosa
API-03	Actualizar producto	actualizarProducto(1, {...})	Llama PUT / productos/1.	Exitosa
API-04	Eliminar producto	eliminarProducto(1)	Llama DELETE / productos/1.	Exitosa
API-05	Error de la API	Respuesta 404 con mensaje	Lanza el mensaje de error.	Exitosa
API-06	Siguiente id	[{id:1},{id:7},{id:3}]	Devuelve 8; con arreglo vacío devuelve 1.	Exitosa
API-07	Error de conexión	TypeError("Failed to fetch")	Mensaje amigable que sugiere npm run mock.	Exitosa

5.2 tests/carrito.test.js — carrito

ID	Caso	Entrada	Resultado esperado	Estado
CAR-01	Agregar producto	Producto con precio base y oferta	Usa el precio de oferta y cantidad 1.	Exitosa
CAR-02	Sumar cantidades	Agregar dos veces el mismo producto	Una línea con cantidad 2.	Exitosa
CAR-03	Total exacto	2 unidades de $1.710	Total $3.420.	Exitosa
CAR-04	Límite de stock	Cambiar cantidad a 99 con stock 10	Cantidad 10 y total $17.100.	Exitosa
CAR-05	Cantidad mínima	Cambiar cantidad a 0	Cantidad 1.	Exitosa
CAR-06	Quitar producto	quitarDelCarrito(1)	Carrito vacío.	Exitosa
CAR-07	Contar unidades	3 unidades	Devuelve 3.	Exitosa
CAR-08	Vaciar carrito	vaciarCarrito()	Arreglo vacío.	Exitosa

5.3 tests/validaciones.test.js — validaciones

ID	Caso	Entrada	Resultado esperado	Estado
VAL-01	RUN válidos	12345678-5, 21605460-1, 12.345.678-5	Verdaderos.	Exitosa

VAL-02	RUN inválidos	12345678-9, sin-guion	Falsos.	Exitosa
VAL-03	Cálculo de DV	12345678, 21605460	5 y 1.	Exitosa
VAL-04	Separar y normalizar	12.345.678-5	{run:12345678, dvrun:"5"}.	Exitosa
VAL-05	Correo	cliente@correo.cl / sin arroba	Verdadero / falso.	Exitosa
VAL-06	Teléfono	912345678 / 1234	Verdadero / falso.	Exitosa
VAL-07	Texto sin números	Carolina / C4rolina	Verdadero / falso.	Exitosa
VAL-08	Usuario y contraseña	vendedor_1, 123	Verdaderos; casos cortos falsos.	Exitosa

5.4 tests/Boton.test.js — componente Boton

ID	Caso	Entrada	Resultado esperado	Estado
BOT-01	Render y clic	<Boton onClick={spy}>	Renderiza y llama al handler una vez.	Exitosa
BOT-02	Deshabilitado	disabled	No dispara el evento y el botón queda deshabilitado.	Exitosa
BOT-03	Variante visual	variante="peligro"	Incluye la clase btn-danger.	Exitosa

5.5 tests/TarjetaProducto.test.js — tarjeta de producto

ID	Caso	Entrada	Resultado esperado	Estado
TAR-01	Render de datos	Producto con oferta	Muestra nombre, categoría, precio base, oferta y -10%.	Exitosa
TAR-02	Agregar al carrito	Clic en “Agregar”	Llama a onAgregar con el producto.	Exitosa
TAR-03	Modo lectura	mostrarBoton={false}	No muestra el botón de agregar.	Exitosa
TAR-04	Sin stock	stock: 0	Botón deshabilitado y texto “Sin stock”.	Exitosa

5.6 tests/FormularioProducto.test.js — formulario

ID	Caso	Entrada	Resultado esperado	Estado
FOR-01	Guardar producto	Datos completos	Llama a onGuardar con nombre, precios, stock y categoría.	Exitosa
FOR-02	Datos obligatorios	Formulario vacío	Muestra “El SKU es obligatorio” y no guarda.	Exitosa
FOR-03	Oferta inválida	Oferta $2.000 con base $1.900	Muestra el error y no guarda.	Exitosa

Total: 33 casos, 33 exitosos, 0 fallidos.

6. Resultado de la ejecución

```
Ejecutado: 26-10-2026
Comando: npm run test:jsdom
Entorno: Karma 6.4.4 + Jasmine + jsdom
Resultado: 33 SUCCESS
Fallos: 0
Tiempo: 0,35 s
```

La ejecución principal con ChromeHeadless está configurada en npm test. En el entorno de evaluación se usó npm run test:jsdom porque el sandbox bloquea el canal IPC de Chromium. En un computador normal, npm test utiliza Chrome.

7. Matriz de cobertura frente a la rúbrica

Aspecto solicitado	Evidencia
Pruebas de renderizado	TAR-01, TAR-03, TAR-04, BOT-01, FOR-01.
Renderizado condicional	TAR-03 (sin botón), TAR-04 (sin stock), FOR-02/FOR-03 (errores).
Pruebas de props	TAR-02 (producto y handler), BOT-01 (onClick), FOR-01 (onGuardar).
Pruebas de estado	FOR-01, FOR-02, FOR-03 (cambios de campos y errores).
Pruebas de eventos	BOT-01, BOT-02, TAR-02, FOR-01.
CRUD	API-01 a API-05.
Persistencia local	CAR-01 a CAR-08 (carrito en localStorage).
Validaciones	VAL-01 a VAL-08.

8. Cómo ejecutar las pruebas

```
# Ejecución principal con ChromeHeadless
npm test

# Alternativa con jsdom (sin abrir Chrome)
npm run test:jsdom

# Modo interactivo
npm run test:watch
```

Requisitos:

1. Tener instaladas las dependencias con npm install.
2. Para npm test, tener Chrome instalado.
3. No se necesita json-server para las pruebas unitarias, porque fetch se simula con un spy.

9. Limitaciones

- No se automatizaron pruebas E2E de la tienda completa con un navegador real.
- Las pruebas de API simulan fetch; no prueban el servidor json-server de forma integrada.
- Las pruebas de componentes no cubren todas las páginas, sino los componentes y formularios con mayor lógica reutilizable.
- La prueba de ChromeHeadless no se pudo ejecutar dentro del sandbox de esta sesión; sí se ejecutó la misma suite con Karma y jsdom.
- El formulario de checkout y el POS se verificaron de forma manual contra la API simulada (creación de orden, pago, stock y movimiento).

10. Conclusiones

La suite cubre los aspectos centrales de la rúbrica: renderizado, props, estado, eventos, CRUD, persistencia local y validaciones. Los 33 casos permiten detectar regresiones en la lógica del carrito, en las operaciones de la API y en los

componentes reutilizables. Para una siguiente iteración se recomienda agregar pruebas E2E con Playwright o Cypress para los flujos completos de compra y administración.